

★ DevOps Handwritten Notes ★

Aman_views

1. What is DevOps?

→ DevOps is a combination of "Development" (Dev) and "Operations" (Ops). It's a culture, set of practices and tools that helps in continuously delivering high quality software to users.

Why DevOps?

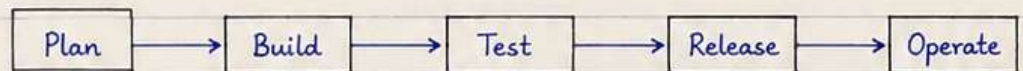
- Bridges the gap between Development and Operations.
- Improves collaboration and communication.
- Automates manual processes and reduces human errors.
- Faster delivery of software.
- Improves reliability, scalability and performance.
- Continuous feedback and improvement.

Dev vs Ops

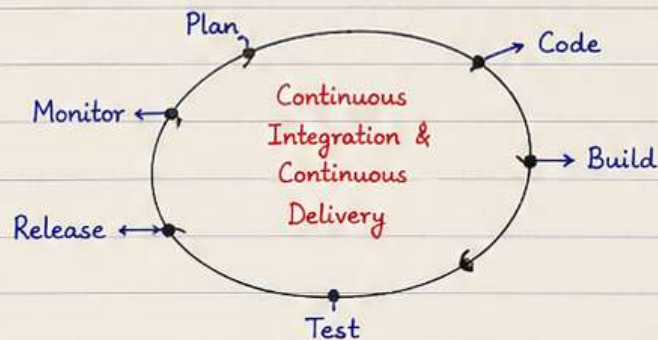
Dev (Development)	Ops (Operations)
• Focus on writing code • New features • Speed of delivery • Works in development environment	• Focus on stability • System reliability • Uptime and performance • Works in production environment

SDLC vs DevOps Lifecycle

Traditional SDLC
(Waterfall)



DevOps Lifecycle
(Continuous)



Benefits of DevOps

- Faster time to market.
- Better collaboration between teams.
- High quality and reliable software.
- Early detection of issues.
- Scalability and automation.

2. CI/CD Pipeline ★

What is CI ?

→ CI (Continuous Integration) is the practice of frequently integrating code changes into a shared repository.

- Ensures early detection of issues.
- Every change is built and tested automatically.

What is CD ?

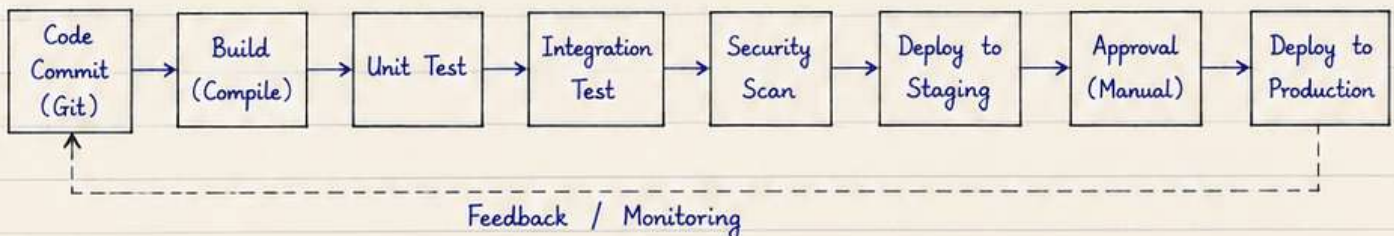
→ CD (Continuous Delivery/Deployment) ensures that code changes are automatically delivered to production safely.

- Continuous Delivery : Code is always ready to be deployed.
- Continuous Deployment : Code is automatically deployed to production.

CD vs Continuous Deployment

Continuous Delivery	Continuous Deployment
• Code is always in a deployable state. • Manual approval required for production. • Safer and more controlled.	• Code is automatically deployed to production. • No manual approval. • Faster but requires strong test coverage.

Typical CI/CD Pipeline Stages



Pipeline Trigger

A pipeline can be triggered in various ways:

1. Git Push

- Developer pushes code to repository.
- Pipeline starts automatically.

2. Pull Request

- When PR is created or updated.
- Runs build & tests before merge.

3. Schedule

- Pipeline runs at specific time.
- Useful for nightly builds, reports etc.

4. Manual Trigger

- Triggered manually by user.
- Useful for re-run, hotfix, etc.

Note

→ A good pipeline should be fast, reliable, and provide quick feedback to developers.

3. Build & Deployment ★

What is Build ?

→ Build is the process of converting source code into an executable artifact that can be tested and deployed.

- Code is compiled.
- Dependencies are resolved.
- Tests are executed.
- Artifact is generated (e.g., .jar, .war, .exe).

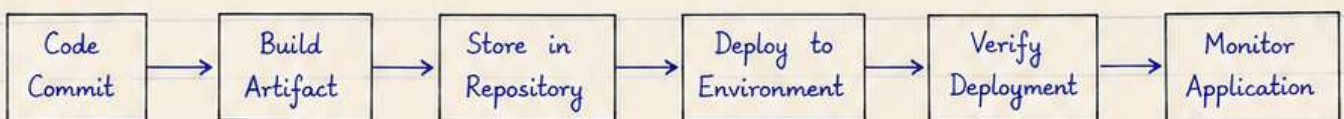
Build Tools

- Maven - Uses pom.xml, follows convention over configuration.
- Gradle - Uses build.gradle, more flexible and faster.

Artifact & Repository

- Artifact - Output of the build process (ready to deploy).
- Repository - Stores artifacts for later use.
→ Example: Nexus Repository, JFrog Artifactory.

Deployment Process



Deployment Strategies

1. Recreate Deployment → Old version is stopped and new version is deployed.
 - Simple but has downtime.
 - Not suitable for high availability.
2. Rolling Deployment → New version is rolled out to servers one by one.
 - No downtime.
 - Safer and commonly used.
3. Blue-Green Deployment → Two environments (Blue & Green). New version is deployed to Green. Traffic is switched from Blue to Green.
 - Zero downtime.
 - Quick rollback.
4. Canary Deployment → New version is released to a small subset of users first.
 - If stable, roll out to all.
 - Reduces risk.

Note

→ Choose deployment strategy based on application criticality, risk tolerance and business needs.

4. Docker & Kubernetes

Docker (Quick Revision)

- Docker is a platform to develop, ship and run applications using containers.
- Container is a lightweight, standalone and executable package.
- Docker Image → Read-only template with instructions.
- Docker Container → Running instance of an image.
- Dockerfile → Script with instructions to build an image.
- Docker Compose → Tool to run multi-container applications.

Docker Flow



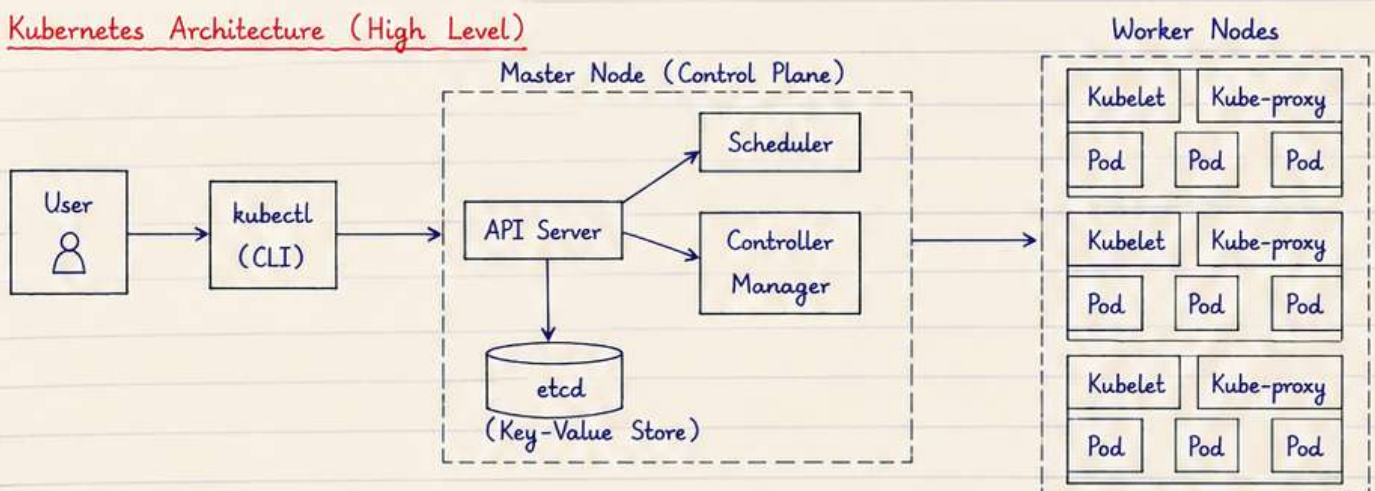
Kubernetes (Quick Revision)

- Kubernetes (K8s) is a container orchestration platform.
- It automates deployment, scaling and management of containerized applications.
- It works with Docker containers.

Key Kubernetes Concepts

- **Pod** → Smallest deployable unit. Contains one or more containers.
- **Deployment** → Manages replica sets and provides declarative updates.
- **ReplicaSet** → Ensures specified number of pod replicas are running.
- **Service** → Provides stable IP and DNS name to access pods.
- **ConfigMap** → Stores configuration data as key-value pairs.
- **Secret** → Stores sensitive information like passwords, tokens.
- **Ingress** → Manages external access to services (HTTP/HTTPS).

Kubernetes Architecture (High Level)



Benefits

- Automated deployment and scaling.
- High availability and self-healing.
- Efficient resource utilization.
- Easy management of containerized applications.

Note → Docker helps in containerization, Kubernetes helps in orchestration.

5. AWS & Cloud ★

What is Cloud ?

→ Cloud Computing is the delivery of computing services (servers, storage, databases, networking, software, analytics, etc.) over the internet ("the cloud") to offer faster innovation, flexible resources and scale.

Why Use Cloud ?

- No need to manage physical hardware.
- Pay for what you use.
- Highly scalable and reliable.
- Access resources from anywhere.
- Faster deployment and better performance.

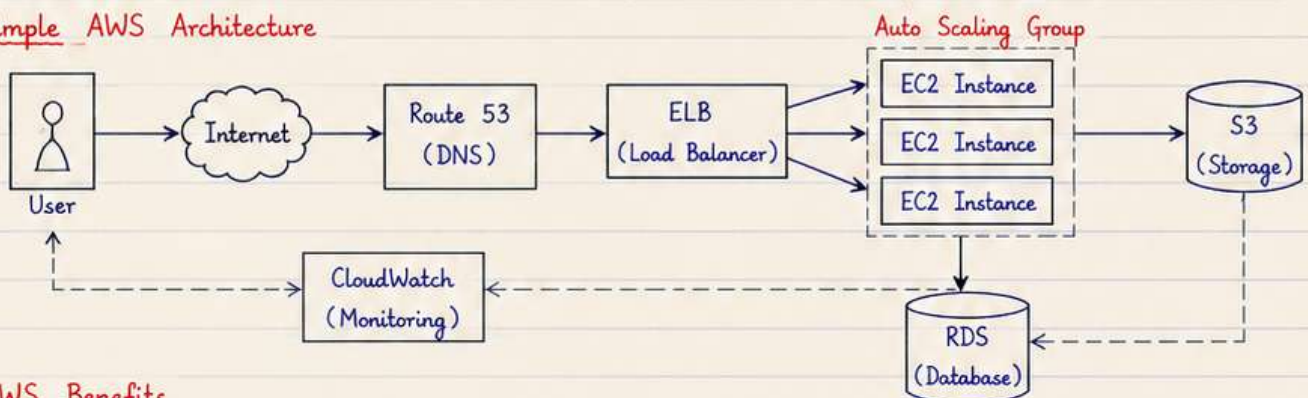
AWS (Amazon Web Services)

→ AWS is a leading cloud platform that provides 200+ fully featured services from data centers globally.

Key AWS Services

Service	Purpose	Example Use
EC2 (Elastic Compute Cloud)	Virtual servers in the cloud	Host applications, backends
S3 (Simple Storage Service)	Object storage	Store files, backups, static content
RDS (Relational Database Service)	Managed relational database	MySQL, PostgreSQL, Oracle, etc.
IAM (Identity & Access Management)	Manage users and permissions	Secure access to AWS resources
ELB (Elastic Load Balancer)	Distribute traffic across servers	High availability applications
Auto Scaling	Automatically adjust instances	Handle traffic spikes
CloudWatch	Monitoring and alerting	Logs, metrics, alarms
VPC (Virtual Private Cloud)	Isolated private network	Secure networking environment
Route 53	DNS service	Domain name management
S3 Glacier	Archival storage	Long-term backup, low cost

Simple AWS Architecture



AWS Benefits

- Highly available and fault tolerant.
- Scalable on demand.
- Pay-as-you-go pricing.
- Secure and compliant.
- Global infrastructure and fast delivery.

Note → Cloud (AWS) provides the infrastructure and services to build, deploy and manage applications without owning physical hardware.

6. Servers, Database & Networking ★

Types of Servers

- Web Server → Handles HTTP requests and serves static content.
- Application Server → Hosts business logic, processes requests and interacts with database.
- Database Server → Stores, manages and provides data.
- Reverse Proxy → Sits in front of servers, forwards requests, provides security and load balancing.

Networking Basics

- IP Address → Unique address to identify a device on a network.
- Domain Name → Human readable name mapped to IP using DNS.
- DNS (Domain Name System) → Translates domain names to IP addresses.
- Port → Logical endpoint to identify a service (e.g., 80, 443, 3306).
- SSL/TLS → Encrypts data in transit to secure communication.
- Firewall → Controls incoming and outgoing traffic for security.

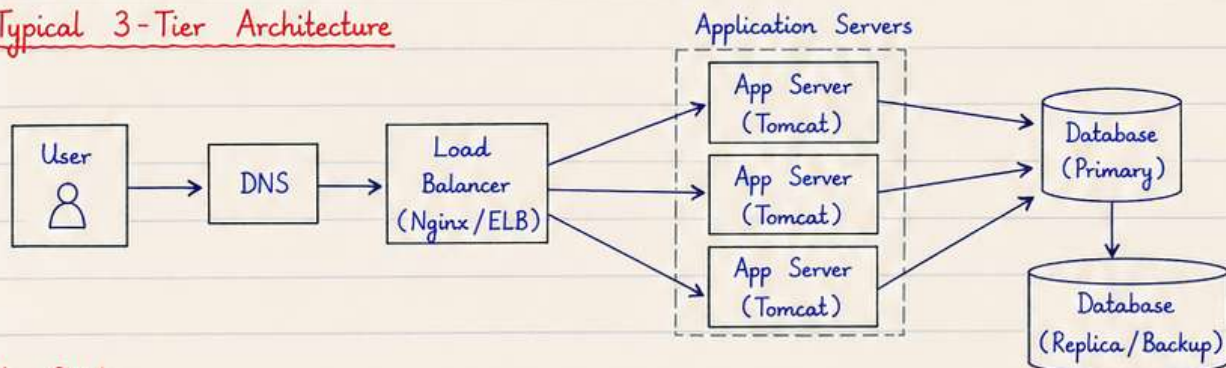
Common Ports

Port	Service	Port	Service
21	FTP	3306	MySQL
22	SSH	5432	PostgreSQL
80	HTTP	8080	HTTP Alternates
443	HTTPS	6379	Redis

Database Concepts

- Primary Database → Main database handling read and write operations.
- Replica (Secondary) → Copy of primary DB, used for read operations and backup.
- Backup → Copy of data stored to recover in case of failure.
- Connection Pool → Reuses existing database connections to improve performance.

Typical 3-Tier Architecture



Key Points

- Use reverse proxy (Nginx) for security, caching and SSL termination.
- Keep database backups regularly.
- Use replicas for high availability and read scaling.
- Monitor server health and network latency.

7. Monitoring & Logging



Why Monitoring?

- Helps to know the health and performance of applications and infrastructure.
- Detects issues early and reduces downtime.
- Improves user experience and system reliability.
- Supports capacity planning and auto scaling.

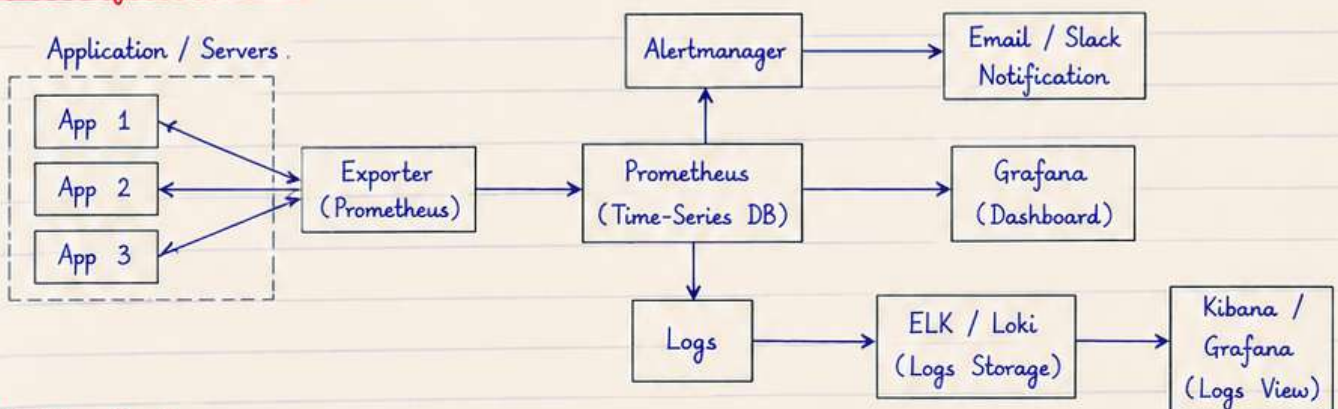
Key Monitoring Concepts

- Metrics → Numerical values that represent system performance (e.g., CPU, Memory).
- Logs → Records of events that happened in the system.
- Traces → Track requests as they flow through the system (Distributed Tracing).
- Alerts → Notifications when something goes wrong or crosses a threshold.
- Dashboards → Visual representation of metrics and logs.

Common Monitoring Tools

Tool	Purpose	Features
Prometheus	Metrics Collection	Time-series DB, Alerting, Pull based
Grafana	Visualization	Dashboards, Graphs, Alerts
ELK Stack (Elasticsearch, Logstash, Kibana)	Logging	Centralized logs, Search, Visualize
Loki	Log Aggregation	Designed for Kubernetes, Cost effective
Splunk	Log Management	Advanced search, Analytics, Alerts

Monitoring Architecture



Health Checks

- Liveness Check → Checks if application is running. If fail, restart the container.
- Readiness Check → Checks if application is ready to serve traffic.
- Startup Check → Checks if application started successfully.

Best Practices

- Monitor everything that matters.
- Set meaningful alerts (not too many, not too few).
- Use dashboards for quick insights.
- Keep logs centralized.
- Retain logs and metrics for analysis.

Note → Good monitoring and logging helps in fast issue detection, root cause analysis and better system reliability.

8. Jenkins, Git & Automation ★

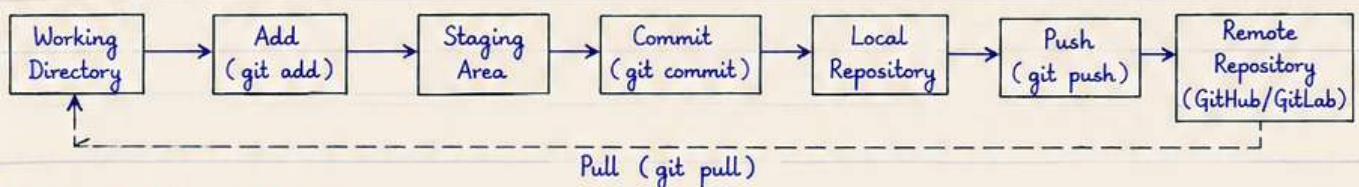
What is Jenkins ?

- Jenkins is an open source automation server.
- It helps to automate the build, test and deployment process.
- It supports Continuous Integration and Continuous Delivery.

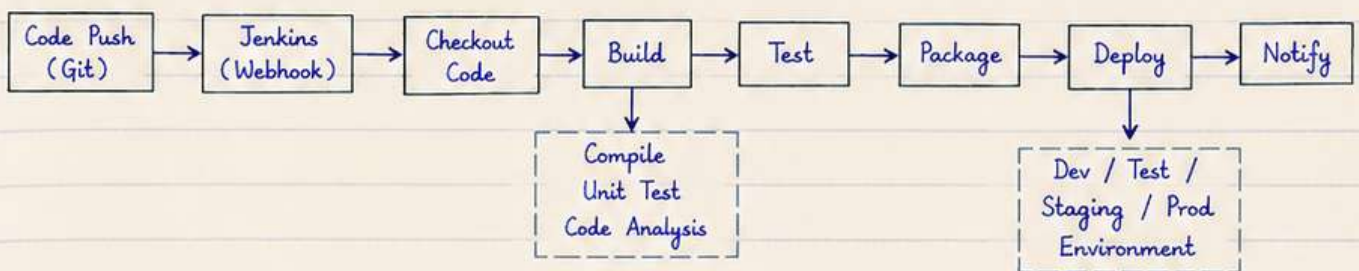
Why Use Jenkins ?

- Automates the manual and repetitive tasks.
- Integrates with multiple tools.
- Extensible using plugins.
- Provides visibility of the entire pipeline.

Git Workflow (Basic)



Jenkins Pipeline (Flow)



Jenkins Job Types

- Freestyle Project → Simple jobs with build steps.
- Pipeline Project → Define entire pipeline as code (Jenkinsfile).
- Multi-branch Pipeline → Automatically creates pipeline for each branch.
- Folder → Organize related jobs.

Important Jenkins Features

- Plugins → Extend Jenkins functionality.
- Credentials → Store usernames, passwords, tokens securely.
- Parameters → Take input at build time.
- Environment Variables → Used to configure builds.
- Build Triggers → Poll SCM, Webhook, Schedule, Manual, etc.

Other Automation Tools

- GitHub Actions → CI/CD from GitHub repository.
- GitLab CI/CD → Built-in CI/CD with GitLab.
- Ansible → Configuration management & automation.
- Terraform → Infrastructure as Code (IaC).

Note → Automation reduces human effort, increases speed and improves reliability.

9. Best Practices & DevOps Culture ☆

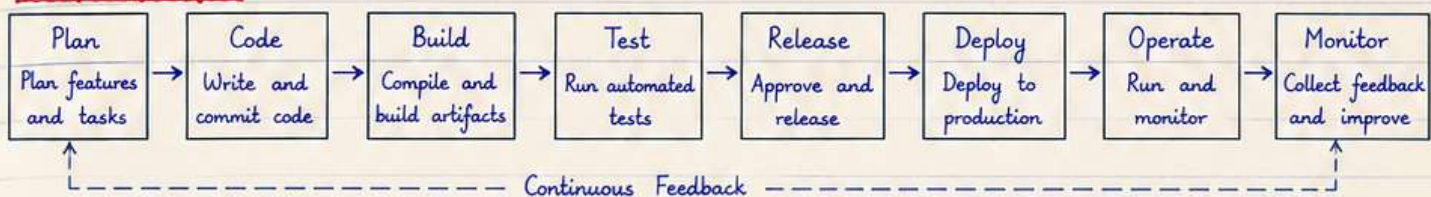
DevOps Best Practices

- Automate Everything → Automate build, test, deploy and monitoring.
- Small & Frequent Releases → Release in small batches, fail fast and fix fast.
- Infrastructure as Code → Manage infrastructure using code (not manual steps).
- Version Everything → Keep code, configs, scripts in version control.
- Continuous Testing → Test early, test often and test automatically.
- Monitor & Feedback → Monitor system, collect feedback and improve.
- Security First → Integrate security checks in the pipeline (DevSecOps).
- Collaboration → Break silos between Dev, Ops, QA and Security teams.

DevOps Culture

- Shared Responsibility → Everyone owns the product from build to production.
- Communication → Open, transparent and frequent communication.
- Trust & Empowerment → Trust teams to make decisions and take ownership.
- Continuous Learning → Learn, experiment and improve continuously.
- Customer Focus → Focus on delivering value to the customer.

DevOps Life Cycle



Common DevOps Tools

Category	Tools (Examples)	Purpose
Source Code Management	Git, GitHub, GitLab, Bitbucket	Store and manage source code
CI Server	Jenkins, GitLab CI, GitHub Actions	Automate build, test and integration
Build Tools	Maven, Gradle, NPM	Compile and package applications
Testing Tools	Selenium, JUnit, Postman, SonarQube	Functional testing, code quality
Artifact Repository	Nexus, Artifactory	Store build artifacts and dependencies
Containerization	Docker	Package application with dependencies
Orchestration	Kubernetes	Deploy, scale and manage containers
Configuration Management	Ansible, Puppet, Chef	Automate infrastructure configuration
Monitoring	Prometheus, Grafana, ELK Stack	Monitor systems and visualize logs

Key Takeaways

- DevOps combines Development and Operations to deliver software faster and reliably.
- Automation, collaboration and continuous feedback are the heart of DevOps.
- Start small, automate, measure and keep improving.
- Tools are important, but culture and mindset make DevOps successful.

Note → DevOps is a journey, not a destination. Keep learning, automating and improving every day.

COMMENT PDF

— TO GET —

PDF NOTES



Aman Views

Java • SQL • System Design • DSA
Backend Development • Web Development
QA Testing

Handwritten Notes



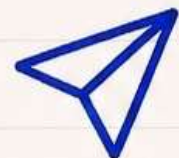
TO GET LINK OF ALL OUR NOTES,
CHECK THE LINK IN BIO.



SAVE



FOLLOW



SHARE